XMP Preset Generation Methods

1. Overview

This document describes two improved methods for generating Adobe Lightroom XMP preset values from image analysis. Both methods replace the original hardcoded scale-factor approach with more accurate techniques.

Available methods:
- basic: Original hardcoded LAB/HSV delta mapping (default)
- color_science: Proper color science via colour-science library
- optimization: Forward model + scipy.optimize

Usage:

```
generator = XMPPresetGenerator()
params = generator.extract_params(target, reference, strength=1.0, method="color_science")
# or method="optimization"
```

2. Color Science Method

Uses the colour-science Python library for physically accurate color calculations instead of hardcoded multipliers.

2.1 Temperature (CCT)

Problem: The basic method uses LAB b-channel shift * 100 to estimate color temperature in Kelvin. This is a rough linear approximation.

Solution: Convert mean image color to CIE XYZ, then to CIE 1960 UCS uv chromaticity, then use the Ohno 2013 method to compute Correlated Color Temperature (CCT). The CCT difference between target and reference images gives the temperature slider value directly in Kelvin.

```
xyz = colour.sRGB_to_XYZ(mean_rgb)
ucs = colour.XYZ_to_UCS(xyz)
uv = colour.UCS_to_uv(ucs)
cct, duv = colour.uv_to_CCT(uv, method='Ohno 2013')
```

2.2 Tint (Duv)

Problem: The basic method uses LAB a-channel shift * 3.5 for tint.

Solution: Duv (distance from the Planckian locus) directly measures the green-magenta deviation, which is exactly what Lightroom's tint slider controls. Duv ranges from about -0.02 to +0.02 in practice; scaling by 7500 maps this to Lightroom's -150 to +150 range.

2.3 Exposure (log2)

Problem: The basic method uses (L_shift / 255) * 6.0 which is linear.

Solution: Photography exposure is logarithmic (each stop = 2x light). Convert LAB L* to relative luminance Y using the CIE definition:

```
Y = ((L* + 16) / 116)^3  for L* > 7.9996
```

Then compute: exposure_EV = log2(Y_ref / Y_target)

This directly gives the exposure difference in EV stops, which is what Lightroom's exposure slider measures.

```
target_Y = l_to_luminance(mean_L_target)
ref_Y = l_to_luminance(mean_L_ref)
exposure_ev = log2(ref_Y / target_Y)
```

2.4 Contrast (std ratio)

Problem: Basic method uses raw difference of L standard deviations.

Solution: Use the ratio of L* standard deviations instead:
  contrast = ((ref_std / target_std) - 1.0) * 100

This is multiplicative rather than additive, matching how Lightroom's contrast slider works (it scales the spread around the midpoint).

2.5 Saturation (CIELAB chroma)

Problem: Basic method uses HSV saturation which is not perceptually uniform.

Solution: Use CIELAB chroma C* = sqrt(a*^2 + b*^2) which is perceptually uniform. Compare the ratio of mean chromas between reference and target.

2.6 Vibrance (low-chroma targeting)

Problem: Basic method computes vibrance as saturation * 0.6.

Solution: Vibrance targets low-saturation pixels more than high-saturation ones. Compute chroma shift for only the bottom 50th percentile of chroma values, giving a more accurate vibrance estimate.

XMP Preset Generation Methods

3. Optimization Method

Uses a simplified forward model of Lightroom's processing pipeline combined with scipy.optimize to find slider values that best reproduce the reference image when applied to the target image.

3.1 Forward Model

A LightroomForwardModel class approximates 14 key Lightroom operations in LAB color space:

1. Exposure: Convert L* to luminance Y, scale by 2^EV, convert back
2. Contrast: Expand/compress L around the midpoint
3. Highlights: Selectively adjust top-quarter luminance
4. Shadows: Selectively adjust bottom-quarter luminance
5. Whites: Adjust extreme high luminance
6. Blacks: Adjust extreme low luminance
7. Tone Curve: Parametric adjustments to 4 luminance zones
8. Temperature: Shift LAB b channel (blue-yellow axis)
9. Tint: Shift LAB a channel (green-magenta axis)
10. Saturation: Scale LAB chroma uniformly
11. Vibrance: Scale chroma with inverse-chroma weighting

3.2 Optimization Process

1. Get initial guess from ColorScienceStrategy (warm start)
2. Downsample both images to 256px max dimension for speed
3. Define cost function: weighted MSE between forward model output and reference image in LAB space (L weighted 2x vs A/B)
4. Run L-BFGS-B optimization with max 200 iterations
5. Merge optimized values for the 14 key sliders with ColorScience values for the remaining 57 sliders (HSL, color grading, sharpening)

```
result = scipy.optimize.minimize(
    cost, x0, method='L-BFGS-B',
    bounds=slider_bounds,
    options={'maxiter': 200, 'ftol': 1e-6}
)
```

3.3 Why L-BFGS-B?

L-BFGS-B is chosen because:
- Supports box constraints (slider min/max bounds)
- Memory-efficient for moderate-dimensional problems (14 params)
- Good convergence on smooth, continuous cost functions
- Much faster than global optimizers for this problem size

3.4 Fallback Behavior

If optimization fails (convergence issues, numerical errors), the method falls back to ColorScienceStrategy results automatically.

XMP Preset Generation Methods

4. Comparison

Aspect	Basic	Color Science	Optimization
Temperature	b_shift * 100	CCT via Ohno 2013	Optimized
Tint	a_shift * 3.5	Duv from Planckian	Optimized
Exposure	L/255 * 6.0	log2(Y_ref/Y_tgt)	Optimized
Contrast	std diff * 2.0	std ratio * 100	Optimized
Saturation	HSV sat diff	CIELAB chroma ratio	Optimized
Vibrance	sat * 0.6	Low-chroma targeting	Optimized
Speed	Fast (<100ms)	Fast (<200ms)	Slower (~1-3s)
Accuracy	Rough approx	Good for key sliders	Best overall
Dependencies	numpy, opencv	+ colour-science	+ scipy

5. When to Use Each Method

basic: Default for backward compatibility. Good enough when exact preset accuracy is not critical.

color_science: Recommended when you need better accuracy without added latency. Best improvement-to-cost ratio.

optimization: Use when preset accuracy is paramount and you can tolerate 1-3 seconds of extra processing time per image.